PACMAN PROJECT REPORT

SUBMITTED BY:

M.AHMAD KHAN
2025-CYS-31

SUBMITTED TO:

SIR SHAHMEER NAWAZ

1. Introduction:

This project is a console-based recreation of the classic **Pac-Man** game, developed using **Java**. The primary objective of this project is to demonstrate core **Object-Oriented Programming (OOP)** principles such as encapsulation, inheritance, and association, while managing real-time game logic in a grid-based environment.

2. System Architecture:

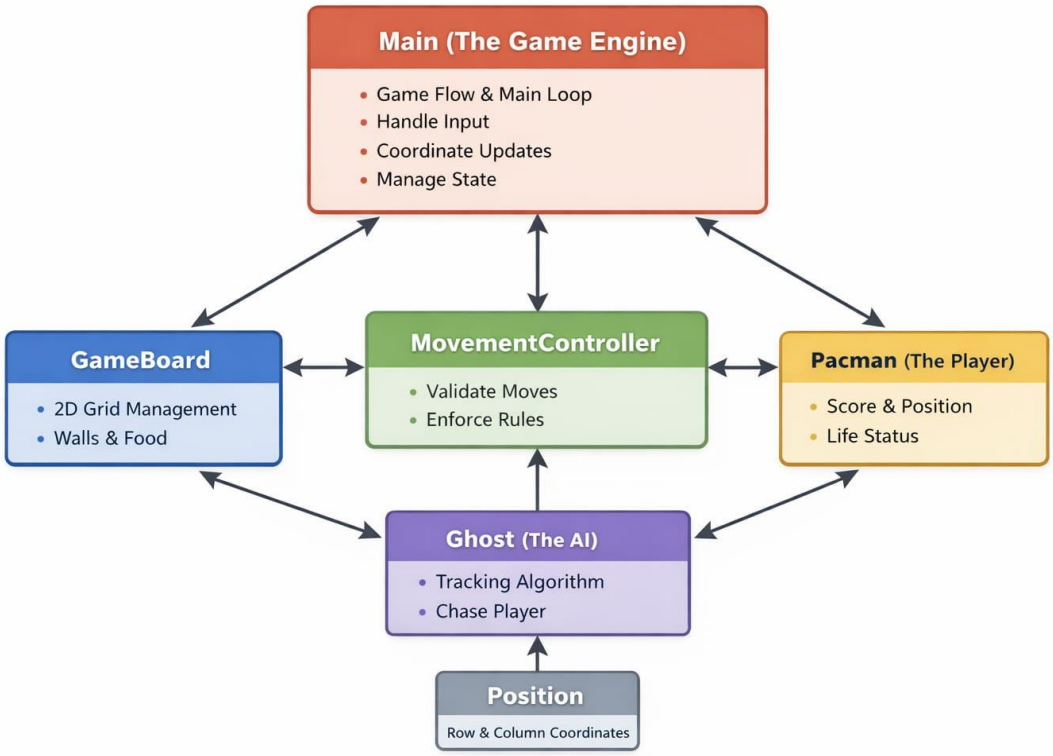
The game follows a modular architecture where each component is isolated into its own class to ensure clean code and scalability.

Classes:

- **Main (The Game Engine):** Orchestrates the game flow, manages the main loop, handles user input, and coordinates updates between the characters and the board.
- **GameBoard:** Defines the 2D grid structure. It manages "walls" (#), "food" (.), and provides methods to render the visual state of the game.
- **Pacman (The Player):** Stores player-specific data such as

score, current position, and life status (isAlive).

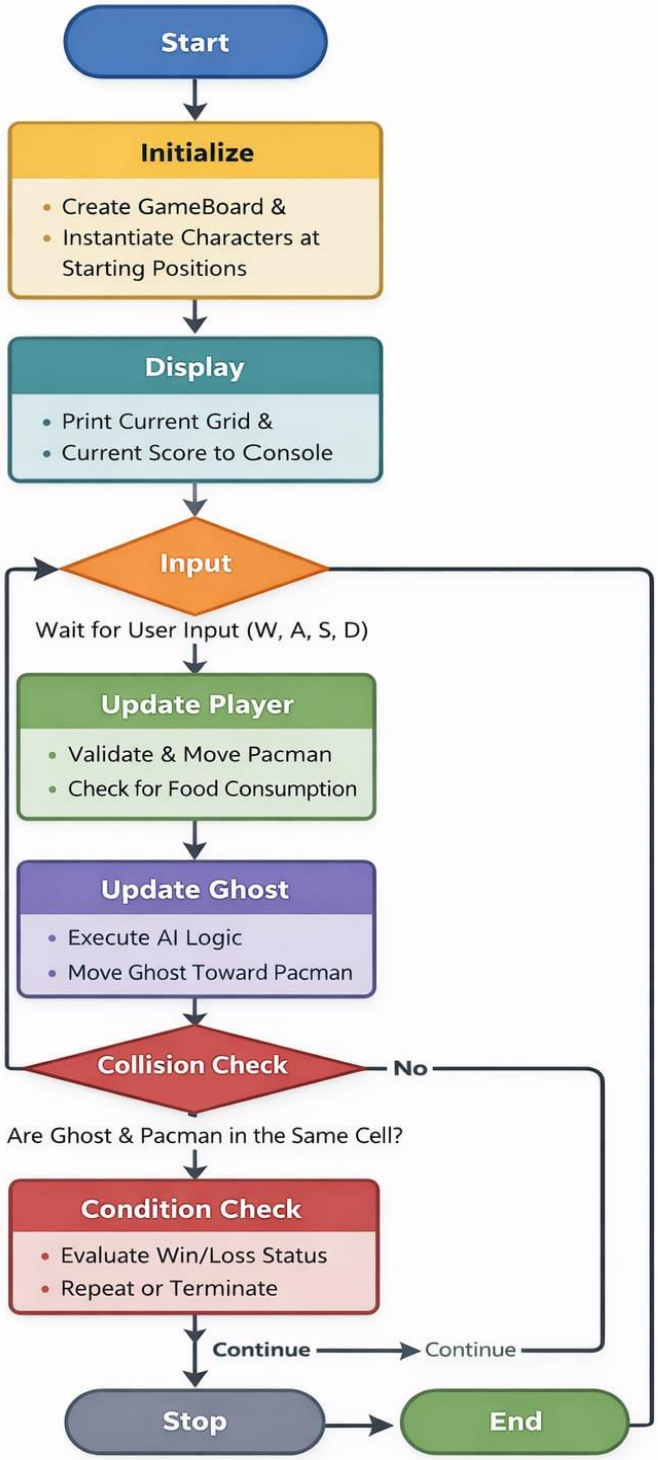
- **Ghost (The AI):** Implements a simple tracking algorithm to follow the player across the grid.
- **MovementController:** Acts as a rule-engine. It checks whether a requested move is valid (i.e., not walking through a wall) before allowing a character to update its position
- **Position:** A helper class used to store and manipulate row and column coordinates.



ARCHITECTURE DIAGRAM

3. Operational Flowchart:

1. **Initialize:** Create the GameBoard and instantiate characters at starting positions.
2. **Display:** Print the current grid and current score to the console.
3. **Input:** Wait for user input (W, A, S, D).
4. **Update Player:** Validate and move Pacman. Check for food consumption.
5. **Update Ghost:** Execute AI logic to move the Ghost toward Pacman.
6. **Collision Check:** Verify if Ghost and Pacman occupy the same cell.
7. **Condition Check:** Evaluate Win/Loss status and repeat or terminate.



Operational Flowchart

4.Ideas for Future:

- **Advanced Pathfinding***:* Implementing the A-Star search algorithm to enable Ghosts to navigate complex maze corners and obstacles more intelligently rather than just moving in a direct line.
- **Finite State Machine (FSM):** Adding game "states" such as **Power-Up Mode**, where ghosts turn vulnerable and flee, and **Scatter Mode**, where they retreat to home corners.
- **Dynamic Level Loading:** Transitioning from a hard-coded grid to a system that reads .txt files, allowing users to create and load custom maze designs without changing the source code.
- **Graphical User Interface (GUI):** Migrating the project from a text-based console to a visual interface (using JavaFX or Swing) to support smooth animations, image sprites, and sound effects.
- **Multi-Ghost Personalities:** Introducing multiple ghosts, each programmed with unique behaviors (e.g., one chases directly, while another tries to intercept the player's path).

5. Conclusion

This project successfully implements a functional game loop using Java OOP. It provides a solid foundation for more complex features, such as multiple ghost levels, power-ups, or a Graphical User Interface (GUI).